

Homework 2

ASEN 5335 Aerospace Environment

Justin Le

September 18, 2026

Problem 1: Spaceballs

(i.)

We are able to use the provided MSISatmosphere1000.m profile to calculate the an approximate total mass of the atmosphere. The MSISatmosphere1000 function takes inputs of altitudes and outputs a structure, one of which is mass density which corresponds to that altitude. Using MATLAB we are able to have a vector of altitudes that range to 1000 km, the maximum model input of the MSISatmosphere1000 function. We can then obtain a vector of mass densities ρ . We use the following,

$$m = \rho V$$

$$m = \int \rho dV$$

And for a sphere,

$$m = \int \rho * 4\pi r^2 dr$$

$$\therefore m = 4\pi \int \rho r^2 dr$$

With ρ being a vector and r also being a vector of radius of this atmospheric shell (altitude including radius of the Earth $R_E = 6371$ km), we can integrate within MATLAB to obtain a

$$\boxed{m = 5.297 * 10^{18} \text{ kg}}.$$

(ii.)

Assuming a volume of $V = 10 * 10^3 * 10 * 10^3 * 100 * 10^3 = 1 * 10^{13} \text{ m}^3$, a constant temperature $T = 300 \text{ K}$, molar mass $M = 0.029 \text{ kg/mol}$, and universal gas constant $R = 8.314 \text{ J/(molK)}$. We can use the ideal gas law,

$$PV = \frac{m}{M}$$

Rewrite to solve for pressure P ,

$$P = \frac{m}{MV} RT$$

$$\boxed{P = 4.556 * 10^{10} \text{ Pa}}$$

Problem 2: Atmospheric Scale Height

To calculate the scale height H of air mixture assuming N_2/O_2 mixture at sea level, we use the equation.

$$H = \frac{kT}{mg}$$

Where we assume a constant $T = 300$ K from the problem, and $k = 1.38 * 10^{-23}$ J/K, the Boltzmann constant, $g = 9.81$ m/s², the gravitational constant. We then seek the molecular mass of N_2/O_2 mixture. This can be obtained from the table provided in the HW. We find the atomic mass of the air mixture provided with the % by volume

$$amu_{air} = 28.0134 (.781) + 31.9988 (.209) = 28.566 \text{ g/mol}$$

$$m_{air} = amu_{air} * \frac{1 \text{ kg}}{1000 \text{ g} * 6.02 * 10^{23} \text{ g/mol}}$$

$$m_{air} = 4.745 * 10^{-26} \text{ kg}$$

This process of finding molecular weight and converting it can be repeated for atomic Oxygen, helium, and hydrogen.

$$amu_{oxygen} = (31.9988 \text{ g/mol})/2 = 15.9994 \text{ g/mol}$$

$$amu_{helium} = 4.0026 \text{ g/mol}$$

$$amu_{hydrogen} = 2.01594 \text{ g/mol}$$

$$m_{oxygen} = 2.6577 * 10^{-26} \text{ kg}$$

$$m_{helium} = 6.6488 * 10^{-27} \text{ kg}$$

$$m_{hydrogen} = 3.3487 * 10^{-27} \text{ kg}$$

The scale height equation can be used with each respective molecular mass of the species in question.

$$H_{air} = 8.8938 * 10^3 \text{ m}$$

$$H_{oxygen} = 1.588 * 10^4 \text{ m}$$

$$H_{helium} = 6.347 * 10^4 \text{ m}$$

$$H_{hydrogen} = 1.2602 * 10^5 \text{ m}$$

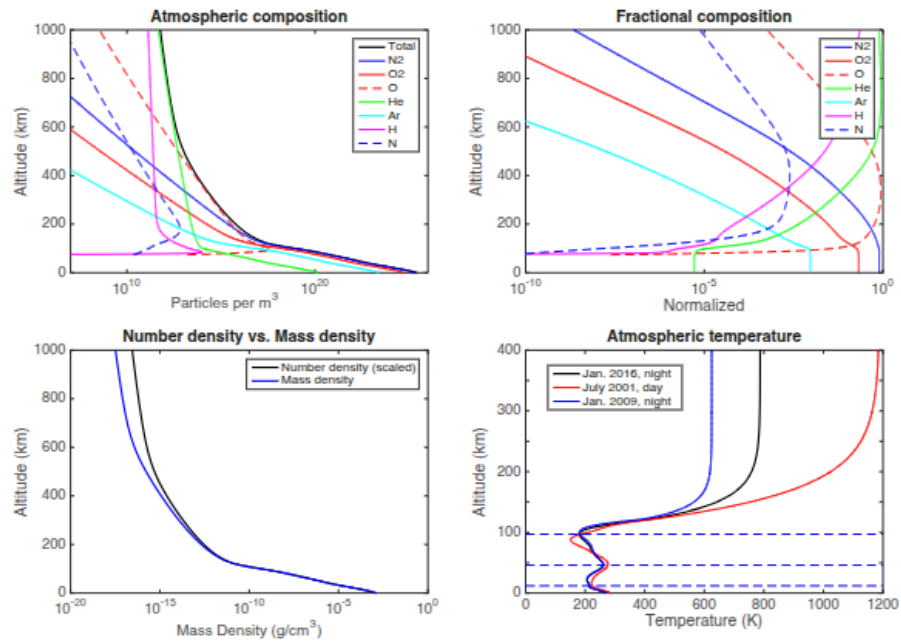


Figure 1: Atmospheric Profile

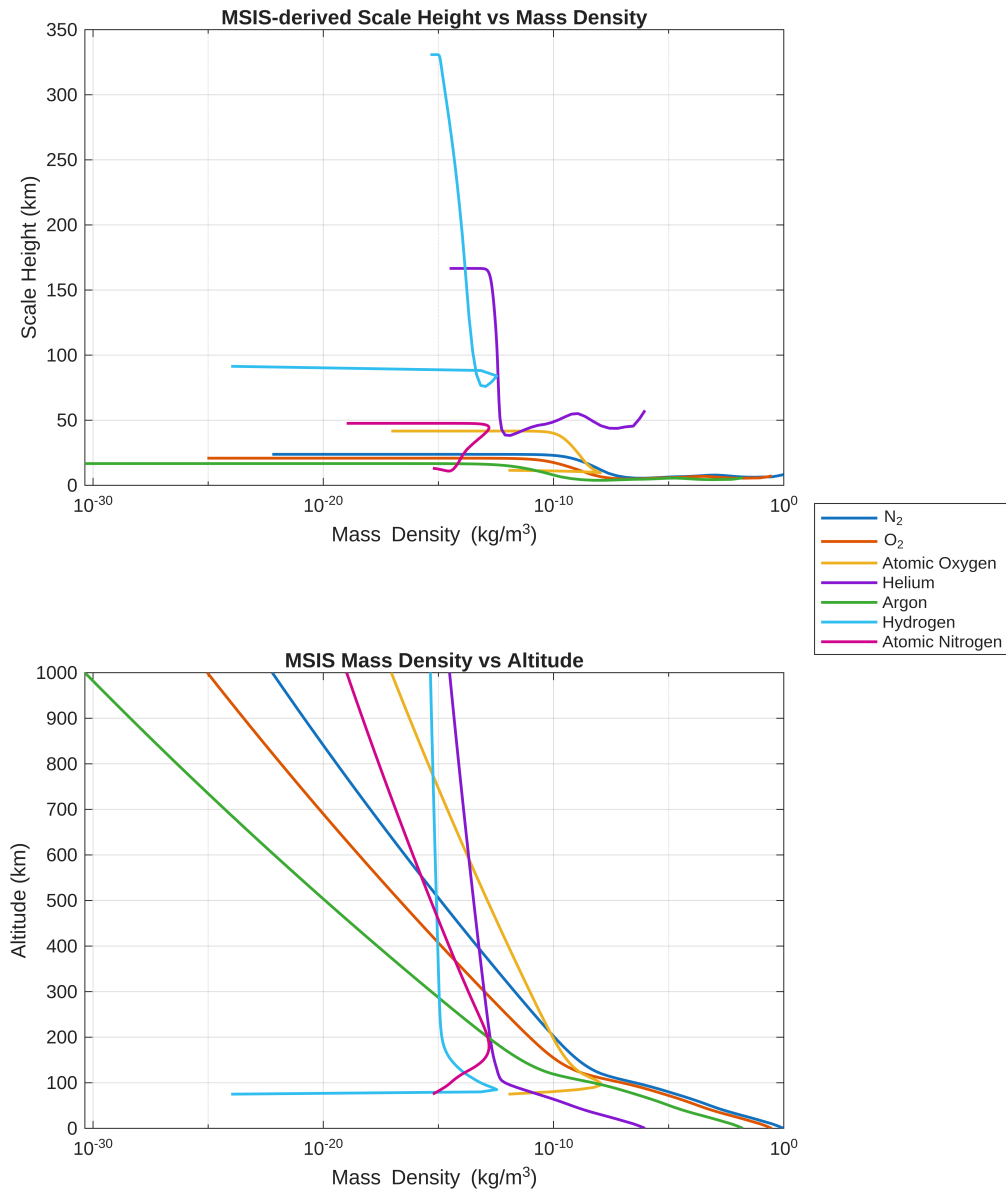


Figure 2: Density vs Scale Height and Altitude of Atmospheric Species

We seek to also calculate scale height from the MSIS profiles. From the MSIS profiles there will be different temperatures instead of the fixed T used earlier. This can be done in MATLAB, which is shown in Figure 2. From here, we can observe that MSIS profile for altitude matches up with the figure provided in the assignment. We can also observe the relation between scale height and the altitudes of each species. Scale height increases as density and altitude increase.

Problem 3: Atmosphere Variation

(i.)

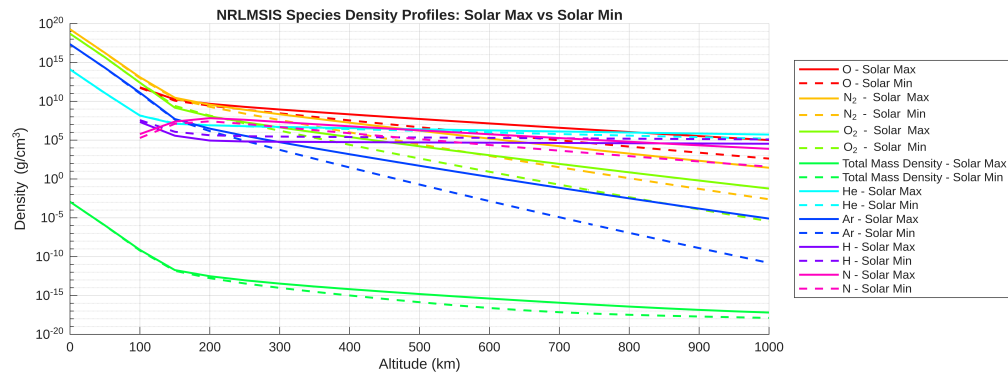


Figure 3: NRLMSIS Species Density Profiles Solar Max vs Solar Min

(ii.)

At 600 km, there is variation between the solar maximum and solar minimum with the density profiles of each species deviating. At solar maximums, the densities of each species tends to be larger by multiple orders of magnitude than they would be a solar minimums. This can even be seen overall with the Total Mass Density variation between solar maximums and minimums. The implication of this for spacecraft operations is that there will be different atmospheric conditions for spacecraft depending on if it is a solar maximum or minimum. For instance, drag will be dependent on the density of the atmosphere, during solar maximums where density can increase by multiple orders of magnitude, drag will also be considerably affected. This could especially impact LEO orbits with their orbits being considerably influenced by drag. Their satellite operations could therefore vary depending on if it is a solar maximum or minimum.

(iii.)

The dominant atmospheric constituent would be O, atomic oxygen, for both solar maximum and solar minimum.

Problem 4: Oxygen Impact

(i.)

To find the characteristic velocity of an oxygen atom O at $T = 1000$ K in the thermosphere, we are able to use the Maxwell Boltzmann Characteristic RMS Velocity equation.

$$v_{rms} = \sqrt{\frac{3k_B T}{m}}$$

$k_B = 1.38 * 10^{-23} \text{ J/K}$ as the Boltzmann constant, and we are able to find $m = 2.657 * 10^{-26} \text{ kg}$ for an oxygen atom. We can plug this into the equation to simply find,

$$v = 1248.26 \text{ m/s}$$

(ii.)

We then calculate the velocity that a spacecraft in LEO at 600 km would have,

$$v_{orbit} = \sqrt{\frac{GM_{Earth}}{r}}$$

$$G = 6.67 * 10^{-11} \text{ Nm}^2/\text{kg}^2$$

$$M_{Earth} = 5.972 * 10^{24} \text{ kg}$$

$$r = 600 * 10^3 \text{ m} + 6371 * 10^3 \text{ m} = 6671 * 10^3 \text{ m}$$

$$\therefore v_{orbit} = 7559 \text{ m/s}$$

The energy imparted onto the spacecraft by an oxygen atom would be using the mass of the oxygen atom and the velocity of the spacecraft,

$$KE = \frac{1}{2} m_{oxygen} v_{orbit}^2$$

$$KE = 7.591 * 10^{-19} \text{ J}$$

Where $1 \text{ J} = 6.242 * 10^{18} \text{ eV}$

$$\therefore KE = 4.738 \text{ eV}$$

(iii.)

Examine the energy threshold for Al being bombarded by O from the table, we find $E_{th} = 23 \text{ eV} = 3.685 * 10^{-18} \text{ J}$. Rewrite the kinetic energy equation to find velocity required for this energy threshold,

$$v_{th} = \sqrt{\frac{2E_{th}}{m_O}}$$

$$v_{th} = 16654.76 \text{ m/s} = 16.654 \text{ km/s}$$

We now want to find the fraction of the Maxwell Boltzmann distribution for atomic oxygen that would have velocities higher than this. We use the diagram provided in the slides to approximate that it would be 0.01% of the atomic oxygen would have sufficient energy to cause sputtering.

Problem 5: Orbit Decay

(i.)

We use the following relations:

$$F_{drag} = \frac{1}{2}\rho V^2 C_D A, \quad V = \sqrt{\frac{GM_E}{a}}, \quad P^2 GM_E = 4\pi^2 a^3$$

Step 1: Drag equation

$$m \frac{dV}{dt} = \frac{1}{2}\rho V^2 C_D A$$

$$\frac{dV}{dt} = \frac{\rho V^2 C_D A}{2m}$$

Step 2: Relate da/dt to dV/dt with the velocity equation

$$V^2 a = GM_E$$

$$\left(\frac{d}{dt}\right) (V^2 a) = \left(\frac{d}{dt}\right) (GM_E)$$

$$2Va \frac{dV}{dt} + V^2 \frac{da}{dt} = 0$$

$$\frac{da}{dt} = -\frac{2a}{V} \frac{dV}{dt}$$

Substituting Step 1,

$$\frac{da}{dt} = -\frac{2a}{V} \left(\frac{\rho V^2 C_D A}{2m} \right)$$

$$\frac{da}{dt} = -\frac{a \rho V C_D A}{m}$$

Step 3: Relate da/dt to dP/dt with Kepler's third law

$$P^2 GM_E = 4\pi^2 a^3$$

$$\left(\frac{d}{dt}\right) (P^2 GM_E) = \left(\frac{d}{dt}\right) (4\pi^2 a^3)$$

$$2P GM_E \frac{dP}{dt} = 12\pi^2 a^2 \frac{da}{dt}$$

$$\frac{da}{dt} = \frac{P GM_E}{6\pi^2 a^2} \frac{dP}{dt}$$

Step 4: Setting the two expressions for da/dt equal,

$$-\frac{a\rho VC_DA}{m} = \frac{P GM_E}{6\pi^2 a^2} \frac{dP}{dt}$$

$$\frac{dP}{dt} = -\frac{6\pi^2 a^3 \rho VC_DA}{m P GM_E}$$

Using $GM_E = V^2 a$ to eliminate GM_E ,

$$\frac{dP}{dt} = -\frac{6\pi^2 a^3 \rho VC_DA}{m P V^2 a}$$

$$\frac{dP}{dt} = -\frac{6\pi^2 a^2 \rho C_DA}{m P V}$$

For a circular orbit, $2\pi a = PV$,

$$\frac{dP}{dt} = -\frac{6\pi^2 a^2}{m (2\pi a)} \rho C_DA$$

$$\frac{dP}{dt} = -\frac{3\pi a \rho C_DA}{m}$$

$$\boxed{\frac{dP}{dt} = -3\pi a \rho \frac{C_DA}{m}}$$

(ii.)

See section at end of assignment for full MATLAB Code.

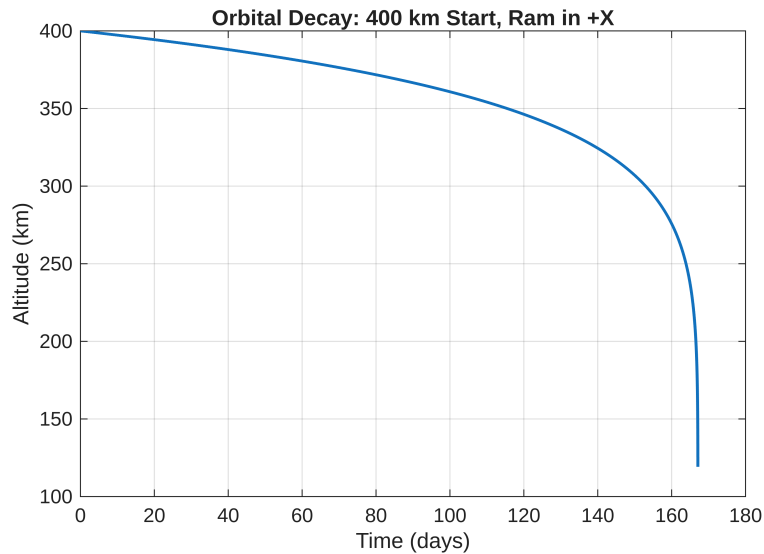


Figure 4: Orbital Decay for 400 km in +x

(iii.)

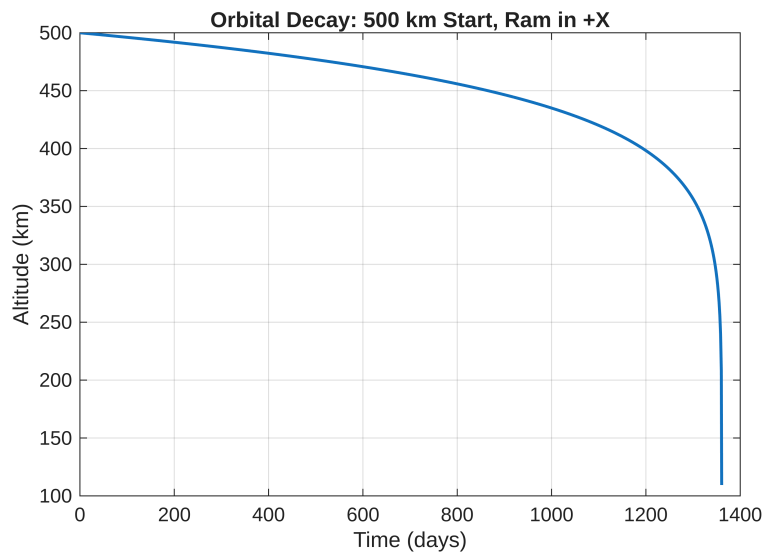


Figure 5: Orbital Decay for 500 km in +x

(iv.)

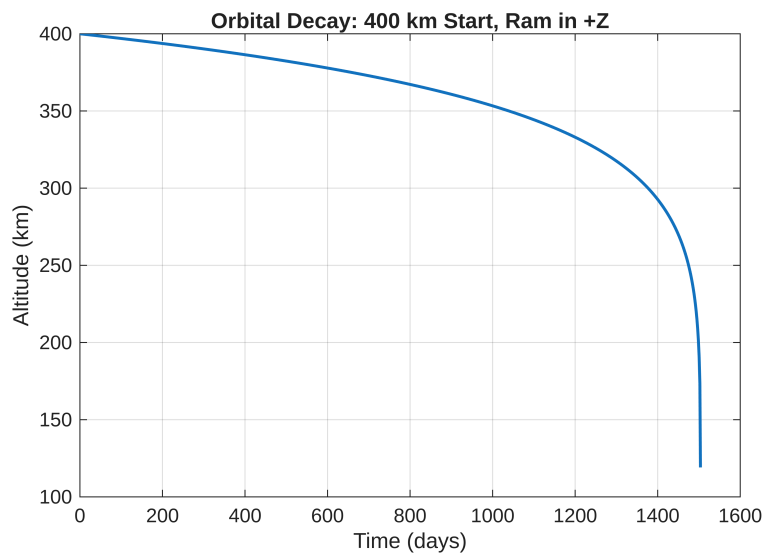


Figure 6: Orbital Decay for 400 km in +z

Problem 6: Atmosphere Uncertainty and Neutral Winds

(i.)

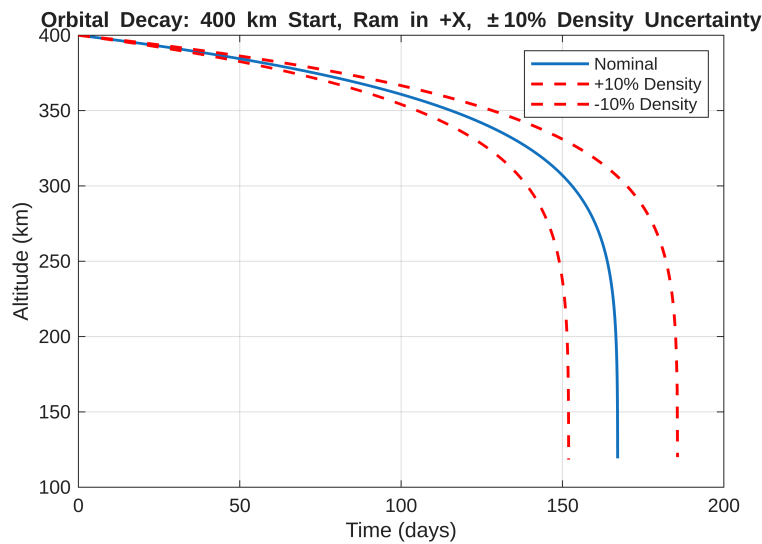


Figure 7: Orbital Decay for 400 km in +z with 10% Uncertainty

We are able to observe a difference in deorbit time of between 33 and 34 days based on the upper and lower bounds of density uncertainty.

(ii.)

Given a 90 minute orbit around Earth at some LEO altitude where this cubesat is operating, the implications for where this spacecraft re entry point in the atmosphere is that there is massive uncertainty in where it will re enter the atmosphere over it's entire ballistic deorbiting process. This location would greatly vary due to this uncertainty.

Problem 7: Air Breathing Thruster

(i.)

In order to find the thrust required to balance the drag force for some spacecraft operating at 400 km, with $C_D = 2.2$ and $A = 100 \text{ cm}^2 = 0.01 \text{ m}^2$, we would start with the drag equation,

$$F_D = \frac{1}{2} \rho V^2 C_D A$$

This means we seek to also solve for velocity and ρ

Solve velocity with the circular orbit relation from Kepler's Law

$$v = \sqrt{\frac{GM_E}{r}}, \text{ where } r = (400 * 10^3 \text{ m} + 6371 * 10^3 \text{ m})$$

$$v = 7700 \text{ m/s}$$

We can obtain the density at 400 km using the MATLAB function MSISatmosphere1000,

$$\rho = 1.194 * 10^{-15} \text{ g/cm}^3 = 1.194 * 10^{-12} \text{ kg/m}^3$$

$$\therefore F_D = 7.787 * 10^{-7} \text{ N} = 0.7787 \mu\text{N}$$

$$\boxed{\therefore F_T = F_D = 0.7787 \mu\text{N}}$$

(ii)

Now to find the exhaust velocity, we can use the relation,

$$F_D = -V_{ex} \frac{dm}{dt}$$

since $\frac{dm}{dt}$ must be equal to the mass collection rate, it is $\frac{dm}{dt} = \rho V A$, at the inlet

$$\therefore \frac{dm}{dt} = 9.194 * 10^{-11} \text{ m/s}$$

$$V_{ex} = -F_D / \frac{dm}{dt}$$

$$\boxed{V_{ex} = -8570 \text{ m/s}}$$

Problem 8: Missions

The method I chose was using space based accelerometer measurements to measure drag and determine atmospheric density. A mission that implemented this was CHAMPS, basically how the method works is that from $F_D = \frac{1}{2}\rho V^2 C_D A$, the drag equation, if you were to understand the vehicle's C_D , A , and V , you can measure the deceleration that the spacecraft is experiencing with accelerometers on board and therefore back out density at some given instance of time. Examining CHAMPS, it was a german spacecraft that was designed to measure Earth's gravity field, magnetic field, and the ionosphere. CHAMPS' accelerometer was located at the center of mass of the spacecraft, and it would be measuring all the non-gravitational accelerations with high resolution. By filtering out the gravitational perturbations, the acceleration that this accelerometer measured would find high resolution mass density of the atmosphere for CHAMPS' mission duration.

MATLAB Code

```

1  %% ASEN 5335 Aerospace Environment
2  %% Justin Le
3  %% HW 2 Problem 1
4
5  % using MSISatmosphere, function provides mass at some given alt
6  % integrate over a range to find total mass
7
8  alt_km = 0:1:1000; % altitude vector
9
10 n = MSISatmosphere1000(alt_km);
11
12 rho_kg_m3 = n.mass*1000; % convert g/cm^3 to kg/m^3
13
14 R_E_m = 6371e3; % Radius of the Earth
15 r_m = R_E_m + alt_km*1000; % distance from Earth's center
16
17 % Integrate with trapz function,  $m = 4\pi \int \rho r^2 dr$ 
18 total_atmos_mass = trapz(r_m, 4*pi*rho_kg_m3.*(r_m.^2)) % kg
19
20 % Part ii
21
22 R = 8.314; % J/molK
23 T = 300; % K
24 M = 0.029; % kg/mol
25 V = 10*10^3*10*10^3*100*10^3; % m^3
26
27 P = (total_atmos_mass/(M*V))*R*T % Pa

```

```

1  %% ASEN 5335 Aerospace Environment
2  %% Justin Le
3  %% HW2 Problem 2
4
5  clc;clear; close all
6
7  % Constants
8  k=1.38E-23;
9  T=300;
10 g=9.81;
11
12 %g/mol to kg conversion
13 gmol2kg = (1/(1000*6.02E23));
14
15 % air
16 amu_air = 28.0124*(0.781) + 31.9988*(0.209);
17 m_air = amu_air*gmol2kg;
18 H_air = (k*T)/(m_air*g)
19 % atomic oxygen
20 amu_oxygen = 31.9988/2;
21 m_oxygen = amu_oxygen*gmol2kg;
22 H_oxygen = (k*T)/(m_oxygen*g)
23 % helium
24 amu_helium = 4.0026;
25 m_helium = amu_helium*gmol2kg;
26 H_helium = (k*T)/(m_helium*g)
27 % hydrogen
28 amu_hydrogen = 2.01594;
29 m_hydrogen= amu_hydrogen*gmol2kg;
30 H_hydrogen = (k*T)/(m_hydrogen*g)
31
32 % Convert to km
33 H_air_km = H_air/1000
34 H_oxygen_km = H_oxygen/1000
35 H_helium_km = H_helium/1000
36 H_hydrogen_km = H_hydrogen/1000
37
38 amu_n2 = 28.0134;
39 amu_o2 = 31.9988;
40 amu_ar = 39.948;
41 amu_n = 14.0067;
42 m_n2 = amu_n2*gmol2kg;
43 m_o2 = amu_o2*gmol2kg;
44 m_ar = amu_ar*gmol2kg;
45 m_n = amu_n*gmol2kg;
46
47 % Some range of altitude
48 alt_range_km = (0:5:1000)';
49 N = length(alt_range_km);
50
51 T_MSIS = zeros(N,1);
52 n_n2_MSIS = zeros(N,1);
53 n_o2_MSIS = zeros(N,1);
54 n_o_MSIS = zeros(N,1);
55 n_he_MSIS = zeros(N,1);
56 n_ar_MSIS = zeros(N,1);
57 n_h_MSIS = zeros(N,1);
58 n_n_MSIS = zeros(N,1);
59

```

```

60 % Loop through entire altitude range and return MSIS information
61 for i = 1:N
62     MSIS_i = MSISatmosphere1000(alt_range_km(i));
63     T_MSI(i) = MSIS_i.temp;
64     n_n2_MSI(i) = MSIS_i.n2;
65     n_o2_MSI(i) = MSIS_i.o2;
66     n_o_MSI(i) = MSIS_i.o;
67     n_he_MSI(i) = MSIS_i.he;
68     n_ar_MSI(i) = MSIS_i.ar;
69     n_h_MSI(i) = MSIS_i.h;
70     n_n_MSI(i) = MSIS_i.n;
71 end
72
73 % Density conversions
74 rho_n2_MSI = n_n2_MSI*1e6*m_n2;
75 rho_o2_MSI = n_o2_MSI*1e6*m_o2;
76 rho_oxygen_MSI = n_o_MSI*1e6*m_oxygen;
77 rho_helium_MSI = n_he_MSI*1e6*m_helium;
78 rho_ar_MSI = n_ar_MSI*1e6*m_ar;
79 rho_hydrogen_MSI = n_h_MSI*1e6*m_hydrogen;
80 rho_n_MSI = n_n_MSI*1e6*m_n;
81
82 % Calculate scale height
83 H_n2_MSI_km = (k*T_MSI)/(m_n2*g)/1000;
84 H_o2_MSI_km = (k*T_MSI)/(m_o2*g)/1000;
85 H_oxygen_MSI_km = (k*T_MSI)/(m_oxygen*g)/1000;
86 H_helium_MSI_km = (k*T_MSI)/(m_helium*g)/1000;
87 H_ar_MSI_km = (k*T_MSI)/(m_ar*g)/1000;
88 H_hydrogen_MSI_km = (k*T_MSI)/(m_hydrogen*g)/1000;
89 H_n_MSI_km = (k*T_MSI)/(m_n*g)/1000;
90
91 % Plotting
92 figure
93 tl = tiledlayout(2,1);
94
95 nexttile
96 semilogx(rho_n2_MSI, H_n2_MSI_km, 'LineWidth', 1.5, 'DisplayName', 'N_2')
97 hold on
98 semilogx(rho_o2_MSI, H_o2_MSI_km, 'LineWidth', 1.5, 'DisplayName', 'O_2')
99 semilogx(rho_oxygen_MSI, H_oxygen_MSI_km, 'LineWidth', 1.5, 'DisplayName', 'Atomic Oxygen')
100 semilogx(rho_helium_MSI, H_helium_MSI_km, 'LineWidth', 1.5, 'DisplayName', 'Helium')
101 semilogx(rho_ar_MSI, H_ar_MSI_km, 'LineWidth', 1.5, 'DisplayName', 'Argon')
102 semilogx(rho_hydrogen_MSI, H_hydrogen_MSI_km, 'LineWidth', 1.5, 'DisplayName', 'Hydrogen')
103 semilogx(rho_n_MSI, H_n_MSI_km, 'LineWidth', 1.5, 'DisplayName', 'Atomic Nitrogen')
104 grid on
105 xlabel('Mass Density (kg/m^3)')
106 ylabel('Scale Height (km)')
107 title('MSIS-derived Scale Height vs Mass Density')
108
109 nexttile
110 semilogx(rho_n2_MSI, alt_range_km, 'LineWidth', 1.5, 'DisplayName', 'N_2')
111 hold on
112 semilogx(rho_o2_MSI, alt_range_km, 'LineWidth', 1.5, 'DisplayName', 'O_2')
113 semilogx(rho_oxygen_MSI, alt_range_km, 'LineWidth', 1.5, 'DisplayName', 'Atomic Oxygen')
114 semilogx(rho_helium_MSI, alt_range_km, 'LineWidth', 1.5, 'DisplayName', 'Helium')
115 semilogx(rho_ar_MSI, alt_range_km, 'LineWidth', 1.5, 'DisplayName', 'Argon')
116 semilogx(rho_hydrogen_MSI, alt_range_km, 'LineWidth', 1.5, 'DisplayName', 'Hydrogen')
117 semilogx(rho_n_MSI, alt_range_km, 'LineWidth', 1.5, 'DisplayName', 'Atomic Nitrogen')
118 grid on

```

```
119 xlabel('Mass Density (kg/m^3)')
120 ylabel('Altitude (km)')
121 title('MSIS Mass Density vs Altitude')
122
123 lgd = legend('Location', 'eastoutside');
124 lgd.Layout.Tile = 'east';
125
126 set(gcf, 'Units', 'inches', 'Position', [0 0 8 10])
127 exportgraphics(gcf, 'HW2Problem2_scaleheight_vs_altitude.png', 'Resolution', 600);
```

```

1  %% ASEN 5335 Aerospace Environment
2  %% Justin Le
3  %% HW2 Problem 3
4
5  clc;clear; close all
6
7  SOLAR_MAX = readmatrix('nrlmsis_output_SOLAR_MAX.txt');
8  SOLAR_MIN = readmatrix('nrlmsis_output_SOLAR_MIN.txt');
9
10 alt_max_km = SOLAR_MAX(:,6); % Height km
11 alt_min_km = SOLAR_MIN(:,6);
12
13 % Reading in species mass densities
14 O_max = SOLAR_MAX(:,9);
15 N2_max = SOLAR_MAX(:,10);
16 O2_max = SOLAR_MAX(:,11);
17 total_mass_max = SOLAR_MAX(:,12);
18 He_max = SOLAR_MAX(:,15);
19 Ar_max = SOLAR_MAX(:,16);
20 H_max = SOLAR_MAX(:,17);
21 N_max = SOLAR_MAX(:,18);
22
23 O_min = SOLAR_MIN(:,9);
24 N2_min = SOLAR_MIN(:,10);
25 O2_min = SOLAR_MIN(:,11);
26 total_mass_min = SOLAR_MIN(:,12);
27 He_min = SOLAR_MIN(:,15);
28 Ar_min = SOLAR_MIN(:,16);
29 H_min = SOLAR_MIN(:,17);
30 N_min = SOLAR_MIN(:,18);
31
32 % NRLMSIS uses 0 or 1e-38 for small densities, filter these out
33 O_max(O_max < 1e-30) = NaN;
34 N2_max(N2_max < 1e-30) = NaN;
35 O2_max(O2_max < 1e-30) = NaN;
36 total_mass_max(total_mass_max < 1e-30) = NaN;
37 He_max(He_max < 1e-30) = NaN;
38 Ar_max(Ar_max < 1e-30) = NaN;
39 H_max(H_max < 1e-30) = NaN;
40 N_max(N_max < 1e-30) = NaN;
41
42 O_min(O_min < 1e-30) = NaN;
43 N2_min(N2_min < 1e-30) = NaN;
44 O2_min(O2_min < 1e-30) = NaN;
45 total_mass_min(total_mass_min < 1e-30) = NaN;
46 He_min(He_min < 1e-30) = NaN;
47 Ar_min(Ar_min < 1e-30) = NaN;
48 H_min(H_min < 1e-30) = NaN;
49 N_min(N_min < 1e-30) = NaN;
50
51 colors = hsv(8);
52
53 figure
54 set(gca, 'YScale', 'log'); % plot as log plot
55 hold on
56 semilogy(alt_max_km, O_max, '-.', 'Color', colors(1,:), 'LineWidth', 1.5, 'DisplayName', 'O - Solar
    ↳ Max')
57 semilogy(alt_min_km, O_min, '--', 'Color', colors(1,:), 'LineWidth', 1.5, 'DisplayName', 'O - Solar
    ↳ Min')

```

```

58 semilogy(alt_max_km, N2_max, '-', 'Color', colors(2,:), 'LineWidth', 1.5, 'DisplayName', 'N_2 -
    ↪ Solar Max')
59 semilogy(alt_min_km, N2_min, '--', 'Color', colors(2,:), 'LineWidth', 1.5, 'DisplayName', 'N_2 -
    ↪ Solar Min')
60 semilogy(alt_max_km, O2_max, '-', 'Color', colors(3,:), 'LineWidth', 1.5, 'DisplayName', 'O_2 -
    ↪ Solar Max')
61 semilogy(alt_min_km, O2_min, '--', 'Color', colors(3,:), 'LineWidth', 1.5, 'DisplayName', 'O_2 -
    ↪ Solar Min')
62 semilogy(alt_max_km, total_mass_max, '-', 'Color', colors(4,:), 'LineWidth', 1.5, 'DisplayName', '
    ↪ Total Mass Density - Solar Max')
63 semilogy(alt_min_km, total_mass_min, '--', 'Color', colors(4,:), 'LineWidth', 1.5, 'DisplayName', '
    ↪ Total Mass Density - Solar Min')
64 semilogy(alt_max_km, He_max, '-', 'Color', colors(5,:), 'LineWidth', 1.5, 'DisplayName', 'He -
    ↪ Solar Max')
65 semilogy(alt_min_km, He_min, '--', 'Color', colors(5,:), 'LineWidth', 1.5, 'DisplayName', 'He -
    ↪ Solar Min')
66 semilogy(alt_max_km, Ar_max, '-', 'Color', colors(6,:), 'LineWidth', 1.5, 'DisplayName', 'Ar -
    ↪ Solar Max')
67 semilogy(alt_min_km, Ar_min, '--', 'Color', colors(6,:), 'LineWidth', 1.5, 'DisplayName', 'Ar -
    ↪ Solar Min')
68 semilogy(alt_max_km, H_max, '-', 'Color', colors(7,:), 'LineWidth', 1.5, 'DisplayName', 'H - Solar
    ↪ Max')
69 semilogy(alt_min_km, H_min, '--', 'Color', colors(7,:), 'LineWidth', 1.5, 'DisplayName', 'H - Solar
    ↪ Min')
70 semilogy(alt_max_km, N_max, '-', 'Color', colors(8,:), 'LineWidth', 1.5, 'DisplayName', 'N - Solar
    ↪ Max')
71 semilogy(alt_min_km, N_min, '--', 'Color', colors(8,:), 'LineWidth', 1.5, 'DisplayName', 'N - Solar
    ↪ Min')
72 grid on
73 xlabel('Altitude (km)')
74 ylabel('Density (g/cm^3)')
75 title('NRLMSIS Species Density Profiles: Solar Max vs Solar Min')
76 legend('Location', 'eastoutside')
77
78 set(gcf, 'Units', 'inches', 'Position', [0 0 12 4]) % smaller physical size for the doc
79 exportgraphics(gcf, 'NRLMSIS_Species_Density_Profiles.png', 'Resolution', 600);

```

```

1  %% ASEN 5335 Aerospace Environment
2  %% Justin Le
3  %% HW2 Problem 5
4
5  clc;clear; close all
6
7  % Constants
8  G = 6.67e-11;
9  M_E = 5.972e24;
10 R_E_m = 6371e3;
11 C_D = 2.2;
12 m_sat_kg = 4; % From earlier in notes
13 A_x_m2 = 0.09;
14 A_z_m2 = 0.01;
15 stop_alt_km = 120;
16
17 % Part ii, 400 km altitude in +x direction
18 [t2_days, alt2_km] = simulateDecay(400, A_x_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat_kg);
19 % Part iii, 500 km in +x direction
20 [t3_days, alt3_km] = simulateDecay(500, A_x_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat_kg);
21 % Part iv, 400 km altitude in +z direction
22 [t4_days, alt4_km] = simulateDecay(400, A_z_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat_kg);
23
24 figure
25 plot(t2_days, alt2_km)
26 grid on
27 xlabel('Time (days)')
28 ylabel('Altitude (km)')
29 title('Orbital Decay: 400 km Start, Ram in +X')
30 set(gcf, 'Units', 'inches', 'Position', [0 0 6 4])
31 exportgraphics(gcf, 'HW2Problem5_OrbitDecay_400km_x.png', 'Resolution', 600);
32
33 figure
34 plot(t3_days, alt3_km)
35 grid on
36 xlabel('Time (days)')
37 ylabel('Altitude (km)')
38 title('Orbital Decay: 500 km Start, Ram in +X')
39 set(gcf, 'Units', 'inches', 'Position', [0 0 6 4])
40 exportgraphics(gcf, 'HW2Problem5_OrbitDecay_500km_x.png', 'Resolution', 600);
41
42 figure
43 plot(t4_days, alt4_km)
44 grid on
45 xlabel('Time (days)')
46 ylabel('Altitude (km)')
47 title('Orbital Decay: 400 km Start, Ram in +Z')
48 set(gcf, 'Units', 'inches', 'Position', [0 0 6 4])
49 exportgraphics(gcf, 'HW2Problem5_OrbitDecay_400km_z.png', 'Resolution', 600);
50
51 function [t_days, alt_km] = simulateDecay(alt0_km, A_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat)
52 % Calculate orbital elements at this time point
53 a = R_E_m + alt0_km*1000;
54 P = 2*pi*sqrt(a^3/(G*M_E));
55
56 % Conditions to iterate
57 t = 0;
58 altitude_index = 1;
59 t_hist(altitude_index) = 0;

```

```
60 alt_hist(altitude_index) = alt0_km;
61
62 % At start, alt = alt0.
63 alt_current_km = alt0_km;
64 % Loop until sat deorbits
65 while alt_current_km > stop_alt_km
66     % Find density at that altitude
67     MSIS = MSISatmosphere1000(alt_current_km);
68     rho = MSIS.mass*1000;
69
70     % Equation 10 and find change in Period
71     dPdt = -3*pi*a*rho*C_D*A_m2/m_sat;
72
73     dt = 600;
74
75     P = P + dPdt*dt;
76     t = t + dt;
77
78     a = (G*M_E*P^2/(4*pi^2))^(1/3);
79     alt_current_km = (a - R_E_m)/1000
80
81     altitude_index = altitude_index + 1;
82     t_hist(altitude_index) = t/86400; % Convert to days
83     alt_hist(altitude_index) = alt_current_km;
84 end
85
86 t_days = t_hist;
87 alt_km = alt_hist;
88
89 end
```

```

1  %% ASEN 5335 Aerospace Environment
2  %% Justin Le
3  %% HW2 Problem 6
4
5  clc;clear; close all
6
7  % Constants
8  G = 6.67e-11;
9  M_E = 5.972e24;
10 R_E_m = 6371e3;
11 C_D = 2.2;
12 m_sat_kg = 4; % From earlier in notes
13 A_x_m2 = 0.09;
14 stop_alt_km = 120;
15 P_orbit_s = 90*60; % Given ~90 min LEO orbital period
16
17 % Nominal case (Problem 5, first case)
18 [t_nom_days, alt_nom_km] = simulateDecay(400, A_x_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat_kg,
    ↪ 1.0);
19 % +10% density
20 [t_hi_days, alt_hi_km] = simulateDecay(400, A_x_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat_kg, 1.1)
    ↪ ;
21 % -10% density
22 [t_lo_days, alt_lo_km] = simulateDecay(400, A_x_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat_kg, 0.9)
    ↪ ;
23
24 figure
25 plot(t_nom_days, alt_nom_km, 'LineWidth', 1.5)
26 hold on
27 plot(t_hi_days, alt_hi_km, 'r--', 'LineWidth', 1.5)
28 plot(t_lo_days, alt_lo_km, 'r--', 'LineWidth', 1.5)
29 grid on
30 xlabel('Time (days)')
31 ylabel('Altitude (km)')
32 title('Orbital Decay: 400 km Start, Ram in +X, \pm 10% Density Uncertainty')
33 legend('Nominal', '+10% Density', '-10% Density')
34 set(gcf, 'Units', 'inches', 'Position', [0 0 6 4])
35 exportgraphics(gcf, 'HW2Problem6_OrbitDecay_400km_x_uncertainty.png', 'Resolution', 600);
36
37 % Calculations for part ii
38 deorbit_day_range = t_lo_days(end) - t_hi_days(end)
39 num_orbits_uncertainty = (deorbit_day_range*86400)/P_orbit_s
40 earth_rotation_per_orbit_deg = (P_orbit_s/86400)*360
41 groundtrack_drift_deg = num_orbits_uncertainty*earth_rotation_per_orbit_deg
42 num_earth_rotations_uncertainty = groundtrack_drift_deg/360
43
44 function [t_days, alt_km] = simulateDecay(alt0_km, A_m2, stop_alt_km, G, M_E, R_E_m, C_D, m_sat,
    ↪ rho_factor)
45 % Calculate orbital elements at this time point
46 a = R_E_m + alt0_km*1000;
47 P = 2*pi*sqrt(a^3/(G*M_E));
48
49 % Conditions to iterate
50 t = 0;
51 altitude_index = 1;
52 t_hist(altitude_index) = 0;
53 alt_hist(altitude_index) = alt0_km;
54
55 % At start, alt = alt0.

```

```
56 alt_current_km = alt0_km;
57 % Loop until sat deorbits
58 while alt_current_km > stop_alt_km
59     % Find density at that altitude
60     MSIS = MSISatmosphere1000(alt_current_km);
61     rho = MSIS.mass*1000*rho_factor;
62
63     % Equation 10 and find change in Period
64     dPdt = -3*pi*a*rho*C_D*A_m2/m_sat;
65
66     dt = 600;
67
68     P = P + dPdt*dt;
69     t = t + dt;
70
71     a = (G*M_E*P^2/(4*pi^2))^(1/3);
72     alt_current_km = (a - R_E_m)/1000;
73
74     altitude_index = altitude_index + 1;
75     t_hist(altitude_index) = t/86400; % Convert to days
76     alt_hist(altitude_index) = alt_current_km;
77 end
78
79 t_days = t_hist;
80 alt_km = alt_hist;
81
82 end
```